

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Технології розподілених систем та паралельних обчислень»

Робота №2

Тема «Побудова паралельних алгоритмів»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Дьячук Т.С.

2025

МЕТА

Метою роботи є одержати навички програмування багатопоточних додатків в WIN32 на прикладі алгоритмів модульного піднесення до степені.

ЗАВДАННЯ

Завдання на роботу:

1. Реалізувати бінарний метод модульного піднесення до степені.
2. Реалізувати двухпоточний варіант алгоритму Монтгомері.
3. Реалізувати метод гребеня модульного піднесення до степені.

Для методу гребеня число потоків повинне вводитися із клавіатури під час виконання програми.

4. Порівняти час виконання операції модульного піднесення до степені трьома реалізованими методами.

ВИКОНАННЯ

1 Графи алгоритмів

Бінарний модуль

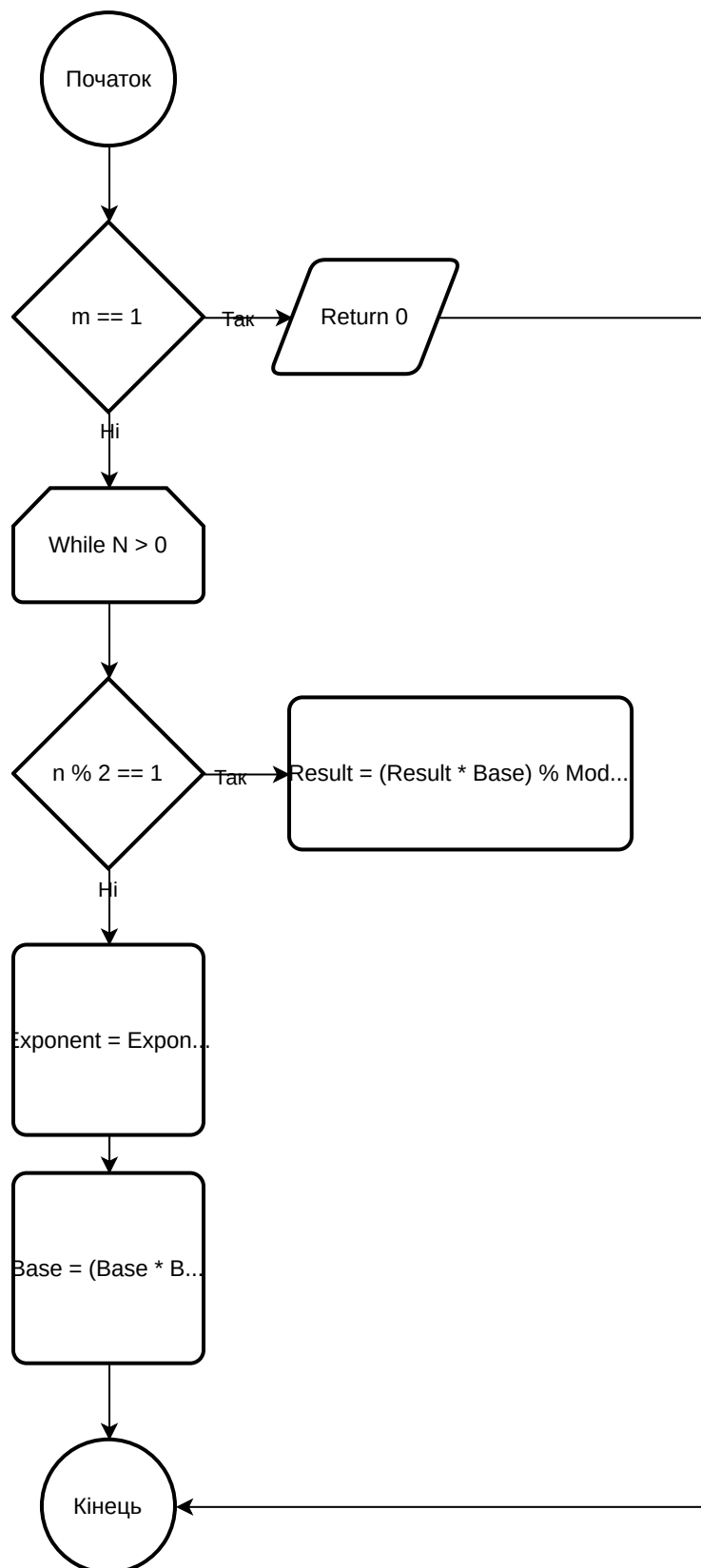


Рисунок 1.1 — Блок-схема бінарного алгоритму

Двухпоточний Монтгомері

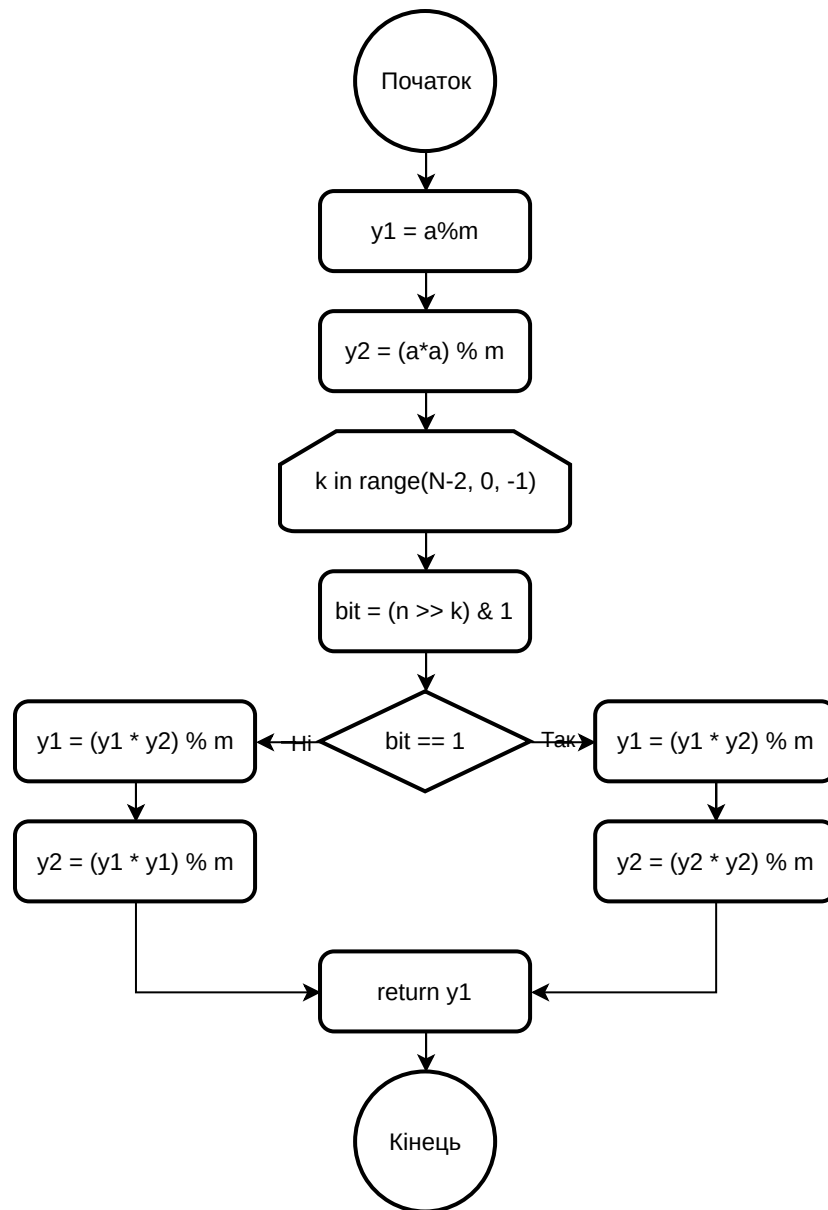


Рисунок 1.2 — Блок-схема алгоритму Монтгомері

Метод гребеня

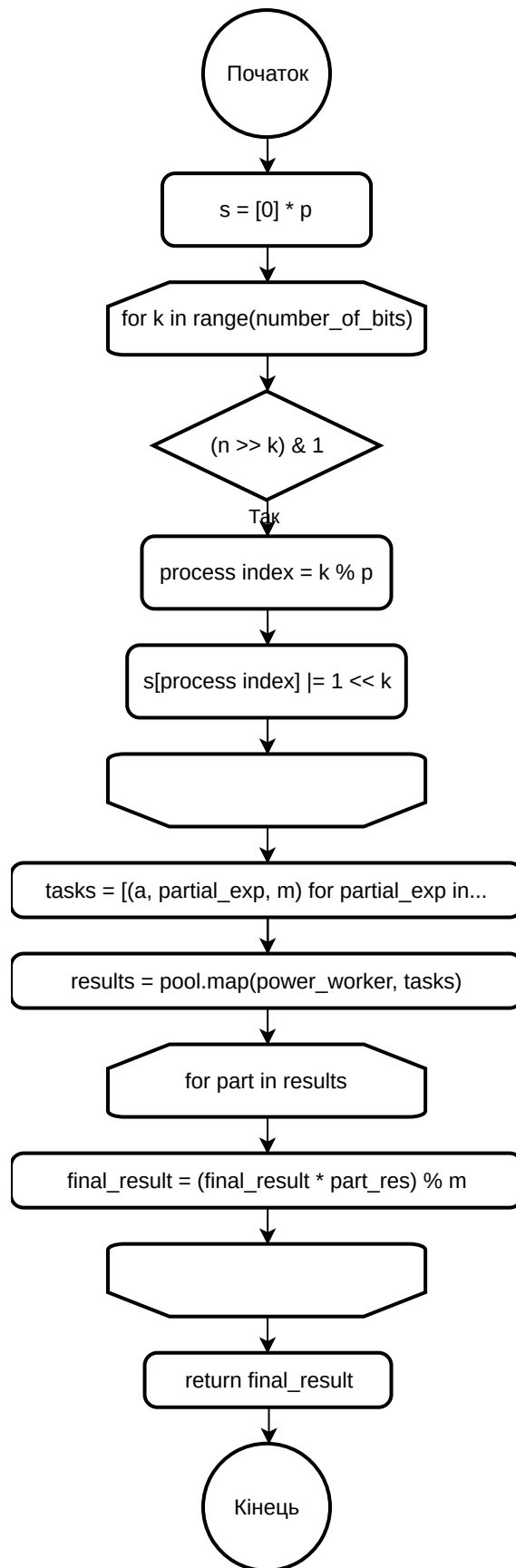


Рисунок 1.3 — Блок-схема алгоритму гребеня

2 Тексти програми

main.py

```
import time
from nicegui import ui
import multiprocessing
from workers import power_worker

with ui.card().classes("mx-auto"):
    ui.label(
        "Так бо Бог полюбив світ, що дав Сина Свого Однородженого, щоб
        кожен, хто вірує в Нього, не згинув, але мав життя вічне. (Йоан 3:16)"
    )

def binary(a: int, n: int, m: int) -> int:
    """
    Бінарний метод піднесення до степені.

    :param a: base
    :type a: int
    :param n: exponent
    :type n: int
    :param m: modulus
    :type m: int
    """

    if n == 0:
        return 1
    if n == 1:
        return a % m

    binary_N = int(bin(n)[2:])
    y = a % m

    for k in range(binary_N - 2, 0, -1):
        y = (y * y) % m

        if (n >> k) & 1:
            y = (y * a) % m

    return y

def montgomery(a: int, n: int, m: int) -> int:
    binary_N = int(bin(n)[2:])

    y1 = a % m
    y2 = (a * a) % m

    for k in range(binary_N - 2, 0, -1):
        bit = (n >> k) & 1

        if bit == 1:
            y1 = (y1 * y2) % m
```

```

        y2 = (y2 * y2) % m
    else:
        y1 = (y1 * y2) % m
        y2 = (y1 * y1) % m

    return y1

def ridge(a: int, n: int, m: int, p: int) -> int:
    if p < 1:
        p = 1

    # 1. Формування часткових показників (гребінь)
    s = [0] * p
    num_bits = n.bit_length()

    for k in range(num_bits):
        if (n >> k) & 1:
            processor_index = k % p
            s[processor_index] |= 1 << k

    # 2. Підготовка завдань
    tasks = [(a, partial_exp, m) for partial_exp in s]

    # 3. Паралельне виконання
    # Важливо: worker імпортовано з зовнішнього файлу
    results = []

    # Використовуємо контекстний менеджер для Pool
    with multiprocessing.Pool(processes=p) as pool:
        results = pool.map(power_worker, tasks)

    # 4. Об'єднання результатів
    final_result = 1
    for part_res in results:
        final_result = (final_result * part_res) % m

    return final_result

def handle_binary(base, exponent, modulus):
    result: int = binary(int(base), int(exponent), int(modulus))
    binary_output.value = result
    binary_output.update()

def handle_montgomery(base, exponent, modulus):
    result: int = montgomery(int(base), int(exponent), int(modulus))
    montgomery_output.value = result
    montgomery_output.update()

def handle_ridge(base, exponent, modulus, processors):
    result: int = ridge(int(base), int(exponent), int(modulus),
int(processors))
    ridge_output.value = result
    ridge_output.update()

```

```

def calculate_times():
    base = 3
    exponent = 10
    modulus = 7
    processors = 4

    binary_start = time.perf_counter()
    binary(base, exponent, modulus)
    binary_end = time.perf_counter()
    binary_time = binary_end - binary_start

    montgomery_start = time.perf_counter()
    montgomery(base, exponent, modulus)
    montgomery_end = time.perf_counter()
    montgomery_time = montgomery_end - montgomery_start

    ridge_start = time.perf_counter()
    ridge(base, exponent, modulus, processors)
    ridge_end = time.perf_counter()
    ridge_time = ridge_end - ridge_start

    data = [
        {"method": TabNames.BINARY, "time": binary_time},
        {"method": TabNames.MONTGOMERY, "time": montgomery_time},
        {"method": TabNames.RIDGE, "time": ridge_time},
    ]
    comparison_table.update_rows(data)

class TabNames:
    BINARY = "Binary"
    MONTGOMERY = "Montgomery"
    RIDGE = "Ridge"
    COMPARISON = "Comparison"

with ui.tabs().classes("w-full") as main_tabs:
    binary_tab = ui.tab(TabNames.BINARY)
    montgomery_tab = ui.tab(TabNames.MONTGOMERY)
    ridge_tab = ui.tab(TabNames.RIDGE)
    comparison_tab = ui.tab(TabNames.COMPARISON)
with ui.tab_panels(main_tabs, value=TabNames.BINARY).classes("w-full"):
    with ui.tab_panel(TabNames.BINARY):
        with ui.row():
            bin_base = ui.input(label="Base")
            bin_exponent = ui.input(label="Exponent")
            bin_modulus = ui.input(label="Modulus")
        ui.button(
            text="Calculate",
            on_click=lambda: handle_binary(
                bin_base.value, bin_exponent.value, bin_modulus.value
            ),
        ).classes("w-full")
        binary_output = ui.input(label="Result").classes("w-full")
    with ui.tab_panel(TabNames.MONTGOMERY):
        with ui.row():
            mont_base = ui.input(label="Base")

```



```

        mont_exponent = ui.input(label="Exponent")
        mont_modulus = ui.input(label="Modulus")
    ui.button(
        text="Calculate",
        on_click=lambda: handle_montgomery(
            mont_base.value, mont_exponent.value,
mont_modulus.value
        ),
    ).classes("w-full")
    montgomery_output = ui.input(label="Result").classes("w-full")
    with ui.tab_panel(TabNames.RIDGE):
        with ui.row():
            ridge_base = ui.input(label="Base")
            ridge_exponent = ui.input(label="Exponent")
            ridge_modulus = ui.input(label="Modulus")
            ridge_processors = ui.input(label="Processors")
        ui.button(
            text="Calculate",
            on_click=lambda: handle_ridge(
                ridge_base.value,
                ridge_exponent.value,
                ridge_modulus.value,
                ridge_processors.value,
            ),
        ).classes("w-full")
        ridge_output = ui.input(label="Result").classes("w-full")
    with ui.tab_panel(TabNames.COMPARISON):
        ui.button(text="Calculate", on_click=lambda: calculate_times())
        cols = [
            {
                "name": "Method",
                "label": "Method",
                "field": "method",
                "required": True,
                "align": "left",
            },
            {
                "name": "Time", "label": "Time", "field": "time",
"required": True},
        ]
        data = [
            {"method": TabNames.BINARY, "time": 0.0},
            {"method": TabNames.MONTGOMERY, "time": 0.0},
            {"method": TabNames.RIDGE, "time": 0.0},
        ]
        comparison_table = ui.table(
            columns=cols,
            rows=data,
            row_key="method",
        )

ui.run(title="TR2", favicon="🏠")

```

workers.py

```

def power_worker(args):
    """

```

```
Ця функція повинна бути в окремому модулі, щоб  
multiprocessing міг її коректно імпортувати (pickle).  
""  
base, exponent, modulus = args  
if exponent == 0:  
    return 1  
# Вбудована функція pow(base, exp, mod) дуже швидка  
return pow(base, exponent, modulus)
```

3 Результати роботи

Бінарний модуль

Так бо Бог полюбив світ, що дав Сина Свого Однородженого, щоб кожен, хто вірує в Нього, не згинув, але мав життя вічне. (Йоан 3:16)

BINARY	MONTGOMERY	RIDGE	COMPARISON
Base 3	Exponent 17	Modulus 7	
CALCULATE			
Result 4			

Рисунок 3.1 — Вигляд результатів роботи

Двухпоточний Монтгомері

Так бо Бог полюбив світ, що дав Сина Свого Однородженого, щоб кожен, хто вірує в Нього, не згинув, але мав життя вічне. (Йоан 3:16)

BINARY

MONTGOMERY

RIDGE

COMPARISON

Base

3

Exponent

17

Modulus

7

CALCULATE

Result

6

Рисунок 3.2 — Вигляд результатів роботи

Метод гребеня

Так бо Бог полюбив світ, що дав Сина Свого Однородженого, щоб кожен, хто вірує в Нього, не згинув, але мав життя вічне. (Йоан 3:16)

BINARY

MONTGOMERY

RIDGE

COMPARISON

Base

3

Exponent

17

Modulus

7

Processors

4

CALCULATE

Result

5

Рисунок 3.3 — Вигляд результатів роботи

4 Гістограма порівняння часу виконання програм

Так бо Бог полюбив світ, що дав Сина Свого Однородженого, щоб кожен, хто вірує в Нього, не згинув, але мав життя вічне. (Йоан 3:16)

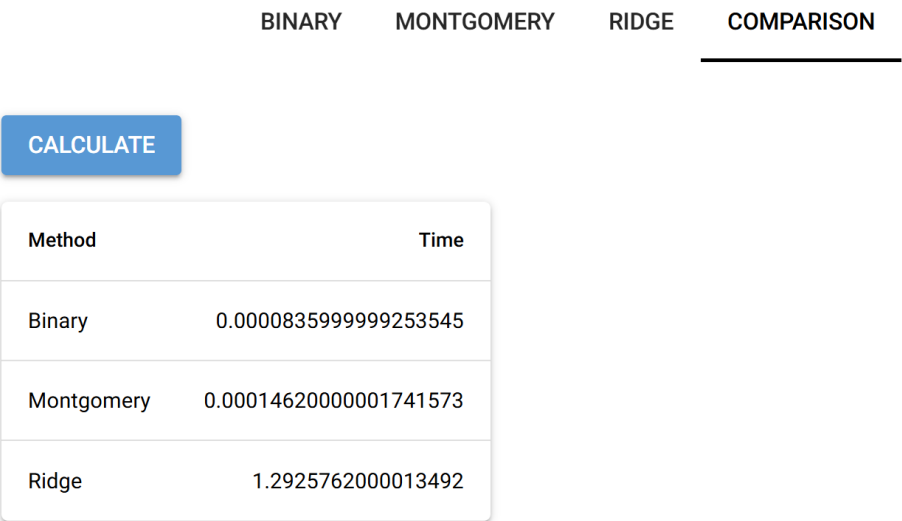


Рисунок 4.1 — Вигляд порівняння результатів

ПИТАННЯ

Методи побудови паралельних алгоритмів. Послідовна та паралельна моделі програмування.

Відомі методи побудови паралельних алгоритмів:

- конвеєрна обробка: процес розбивається на послідовні етапи;
- розділяй та володарюй: задача рекурсивно розділяється на підзадачі, поки вони не стануть достатньо малими для вирішення.

Різниця між послідовною та паралельною моделями програмування наведено на таблиці нижче:

Таблиця 1.1 — Порівняння двох моделей

Характеристика	Послідовна модель	Паралельна модель
----------------	-------------------	-------------------

Виконання	Одна операція за одиницю часу	Багато операцій одночасно
Детермінізм	Результат завжди однаковий	Можливий вплив «стану гонитви» (<i>race condition</i>)
Складність	Проста для розуміння	Висока складність

Парадигми паралельного програмування. Паралелізм даних. Паралелізм задач.

Парадигми паралельного програмування визначають спосіб, у який розробник структурує код для одночасного виконання.

Паралелізм даних — підхід, при якому одна операція одночасно застосовується до різних елементів набору даних.

Паралелізм задач — парадигма, де обчислювальні вузли (потоки / процеси) виконують різні завдання над одними даними.

Як побудувати граф алгоритму?

Для побудови графу алгоритму можна використати вебсайт draw.io. На ньому можна обрати фігури з набору «Блок-схема» та створити граф, перетягуючи елементи на дошку.

З яких кроків складається розробка паралельного алгоритму? В чому сутність кожного кроку

Розробка паралельного алгоритму складається з 4 етапів:

- декомпозиція — поділ великої задачі на менші;

- визначення залежностей — аналіз даних та процесів для виявлення того, які підзадачі потребують результатів інших;
- масштабування — визначення оптимальної кількості підзадач та їх розміру;
- розподіл підзадач — призначення конкретних підзадач на конкретні процесори / ядра.

Джерело:

https://uk.wikipedia.org/wiki/Принципи_розробки_паралельних_методів.